

45 DAY FAANG INTERVIEW PREPARATION PLAN

DSA + Behavioral + Leadership + LLD/HLD Plan

PLAN - PAGE 1

PAGE 1: DSA PLAN (DAYS 1 - 30)

Day	DSA Topic / Pattern	NeetCode Problems
1 - 3	Arrays & Hashing I, II, III	NC 1 - 3, 4 - 6, 7 - 9
4 - 5	Two Pointers I, II	NC 10 - 12, 13 - 15
6	Revision + Easy Problems	Revise & Practice
7 - 9	Sliding Window I, II, III	NC 16 - 18, 19 - 21, 22 - 24
10 - 12	Stack I, II, Monotonic Stack	NC 25 - 27, 28 - 30, 31 - 33
13	Queue / Deque	NC 34 - 36
14 - 15	Binary Search I, II	NC 37 - 39, 40 - 42
16 - 18	Linked List I, II, III	NC 43 - 45, 46 - 48, 49 - 51
19 - 22	Trees I, II, III, IV	NC 52 - 54, 55 - 57, 58 - 60, 61 - 63
23 - 26	BFS I & II, DFS I & II	NC 64 - 66, 67 - 69, 70 - 72, 73 - 75
27	Graphs (All representation/algos)	NC 76 - 78
28	Heap / Priority Queue	NC 79 - 81
29 - 30	Backtracking I, II	NC 82 - 84, 85 - 87

PAGE 2: DSA PLAN (DAYS 31 - 45) + TIMETABLE

Day	DSA Topic / Pattern	NeetCode Problems
31	Backtracking III	NC 88 - 90
32 - 33	Intervals I, II	NC 91 - 93, 94 - 96
34 - 35	Greedy I, II	NC 97 - 99, 100 - 102
36 - 38	Dynamic Programming I, II, III	NC 103 - 105, 106 - 108, 109 - 111
39	DP (Advanced / Knapsack / LCS)	NC 112 - 114
40 - 42	Mixed Practice I, II, III	NC 115 - 117, 118 - 120, 121 - 123
43 - 44	Contest / Hard Problems	NC 124 - 126, 127 - 129
45	Final Revision & Mock Interviews	All Revise

DAILY STUDY TIMETABLE:

- 8:30 - 10:30 PM: Solve 2 Problems (P1 + P2)
- 10:30 - 11:00 PM: Dry Run + Complexity + Explain
- 12:00 - 12:45 AM: Solve 3rd Problem (P3)
- 12:45 - 1:00 AM: Behavioral Topic Practice
- 1:00 - 1:30 AM: LLD / HLD Concept Practice

PAGE 3: BEHAVIORAL + HLD/LLD PLAN

Day	Behavioral / Leadership	LLD / HLD Topic
1-5	Intro, Achievement, Failure, Conflict	SOLID, URL Shortener, Factory
6-10	Leadership, Mentoring, Influence	Rate Limiter, Strategy, Observer
11-15	Customer Focus, Innovation, Priority	Chat System, Singleton, News Feed
16-20	Teamwork, Pressure, Risk Taking	Decorator, Builder, Search Engine
21-25	Prioritization, RCA, Debugging	Payment System, Proxy, Ride Sharing
26-30	Architecture, Trade-offs, Quality	Command, Rate Limiter 2.0, Composite
31-35	Promotion, Manager Rel., Negotiation	State, Distributed Cache, Bridge
36-40	Hiring, Metric Driven, Security	Mediator, Analytics Pipeline, Memento
41-45	Design Thinking, Challenge Story	Visitor, Real-time System, Mock

RESOURCES: NeetCode 150, Grokking the Coding Interview, System Design Primer, Alex Xu System Design.

1. INTERVIEW FLOW (ALWAYS FOLLOW)

1. **Clarify:** Ask constraints, edge cases, duplicates, range, complexity.
2. **Brute Force:** Get it working first ($O(N^2)$).
3. **Better:** Can we optimize? Using HashMap/Set.
4. **Optimal:** Best approach in Time/Space.
5. **Code:** Clean, bug-free code.
6. **Explain:** Walkthrough + dry run.

2. PATTERN RECOGNITION

Need	Use	LC Example
Uniqueness / Duplicate	HashSet	217
Frequency / Count	HashMap / int[26]	242, 347
Pair / Complement	HashMap	1
Grouping	HashMap<Key, List>	49
Top K Frequent	HashMap + Min Heap	347
Left & Right Info	Prefix / Suffix	238

3. PROBLEMS AT A GLANCE

LC #	Problem	Pattern	Key Idea
217	Contains Duplicate	HashSet	If !set.add(x) $\Rightarrow$ duplicate
242	Valid Anagram	Frequency Array	Count++ then Count--
1	Two Sum	HashMap	target - num exists in map?
49	Group Anagrams	HashMap+Freq Sig	26-count key string representation
347	Top K Frequent	HashMap+Heap	Min Heap of size K
271	Encode & Decode	Length+Delimiter	length#string format to encode
238	Product Except Self	Prefix/Suffix	Left[i] * Right[i]

4. JAVA COLLECTIONS & APIS (MOST USED)

HashSet: add(x), contains(x), remove(x), size(), clear()

HashMap: put(k,v), get(k), containsKey(k), getOrDefault(k, def), computeIfAbsent(k, v->newV)

PriorityQueue: offer(x), poll(), peek(), size()

Min Heap Comparator: (a, b) -> a[1] - b[1]

5. CONVERSIONS & BIG-O RULES

Conversions:

- String to Char Array: s.toCharArray()
- Char Array to String: new String(arr)
- Int to String: String.valueOf(x)
- Char to Int: c - '0'

Big-O Rules:

- Sequential loops ADD: $O(N) + O(N) = O(N)$
- Nested loops MULTIPLY: $O(N) * O(N) = O(N^2)$

6. COMMON MISTAKES (AVOID)

- **X** Using sort when HashSet can solve it in $O(N)$.
- **X** Forgetting to check num - 1 in Longest Consecutive Sequence.
- **X** Using split() in Encode/Decode (fails with empty/special characters).
- **X** Not handling negative numbers or edge cases in custom comparator.

CORE PROBLEMS & IMPLEMENTATIONS TABLE

Problem	Key Idea	Approach & Java Code Snippet	Time	Space
217. Contains Duplicate	Insert elements into set. If element exists, return true.	<pre>Set<Integer> set = new HashSet<>(); for (int x : nums) { if (!set.add(x)) return true; // already exists } return false;</pre>	O(N)	O(N)
242. Valid Anagram	Compare character frequency counts.	<pre>int[] count = new int[26]; for (char c : s.toCharArray()) count[c - 'a']++; for (char c : t.toCharArray()) { if (--count[c - 'a'] < 0) return false; } return s.length() == t.length();</pre>	O(N)	O(1)
128. Longest Consecutive	Only start sequence counts from numbers that are the beginning of a sequence (no num-1 in set).	<pre>Set<Integer> set = new HashSet<>(); for (int num : nums) set.add(num); int longest = 0; for (int num : set) { if (!set.contains(num - 1)) { // sequence start int currNum = num, currLen = 1; while (set.contains(currNum + 1)) { currNum++; currLen++; } longest = Math.max(longest, currLen); } }</pre>	O(N)	O(N)

COMMON HASHING FORMULAS & QUICK TRICKS

Top K Frequent Elements (Bucket Sort approach):

```
Map<Integer, Integer> count = new HashMap<>();
for(int n : nums) count.put(n, count.getOrDefault(n, 0) + 1);
List<Integer>[] bucket = new List[nums.length + 1];
for(int key : count.keySet()) {
    int freq = count.get(key);
    if(bucket[freq] == null) bucket[freq] = new ArrayList<>();
    bucket[freq].add(key);
}
```

Encode and Decode Strings (Length-prefix coding):

```
// Encode:
StringBuilder sb = new StringBuilder();
for (String s : strs) sb.append(s.length()).append('#').append(s);
return sb.toString();

// Decode:
List<String> res = new ArrayList<>();
int i = 0;
while (i < s.length()) {
    int slash = s.indexOf('#', i);
    int len = Integer.parseInt(s.substring(i, slash));
    res.add(s.substring(slash + 1, slash + 1 + len));
    i = slash + 1 + len;
}
```

1. FUNDAMENTALS & MIN VS MAX HEAP

Definition: A Heap is a Complete Binary Tree that satisfies the Heap Property.

- **Min Heap:** Parent $\leq$ Children (smallest element at root).
- **Max Heap:** Parent $\geq$ Children (largest element at root).
- **Array Representation:**
 - Parent index: $(i - 1) / 2$
 - Left Child: $2 * i + 1$
 - Right Child: $2 * i + 2$

Time Complexities:

Operation	Min/Max Heap	Why?
Insert (Offer)	$O(\log N)$	Heapify Up
Remove Min/Max (Poll)	$O(\log N)$	Heapify Down
Get Min/Max (Peek)	$O(1)$	Root access
Build Heap	$O(N)$	Bottom-up heapify

2. JAVA PRIORITYQUEUE BASICS

Min Heap (Default):

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
```

Max Heap (3 Ways to implement):

1. Using Collections reverse:

```
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(  
    Collections.reverseOrder());
```

2. Using lambda custom comparator:

```
PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) ->  
    Integer.compare(b, a));
```

3. Using math subtraction (not recommended - overflow risk):

```
PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) -> b - a);
```

Common Operations: offer(x), poll(), peek(), size(), isEmpty().

3. IMPORTANT POINTS & MISTAKES

Key Points to Remember:

- Heaps are for **Priority**, NOT for sorted order! (Iterating elements is not sorted).
- Building heap bottom-up is $O(N)$ which is better than doing N insertions $O(N \log N)$.

Common Mistakes:

- Confusing Min Heap with Max Heap.
- Using subtraction in custom comparator with potential integer overflow edge cases.
- Not checking if heap is empty (isEmpty()) before calling poll() or peek().

HEAP INTERVIEW PATTERNS TABLE

Pattern	When to Use & Examples	Java Template & Key Idea	Complexity
1. Repeated Max/Min	Always pull the largest/smallest element, modify, and put back. Ex: <i>Last Stone Weight</i> .	<pre>PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder()); for(int s : stones) maxHeap.offer(s); while (maxHeap.size() > 1) { int diff = maxHeap.poll() - maxHeap.poll(); if (diff > 0) maxHeap.offer(diff); }</pre>	Time: O(N log N) Space: O(N)
2. Top-K Elements	Keep K largest/smallest elements and discard the rest. Ex: <i>Kth Largest Element</i> .	Min Heap of size K: <pre>PriorityQueue<Integer> pq = new PriorityQueue<>(); for(int x : nums) { pq.offer(x); if(pq.size() > k) pq.poll(); // keeps k largest } return pq.peek();</pre>	Time: O(N log K) Space: O(K)
3. Merge K Sorted	Merge K sorted lists/arrays. Min heap keeps heads of each list. Ex: <i>Merge K Sorted Lists</i> .	<pre>PriorityQueue<ListNode> pq = new PriorityQueue<>((a,b)->a.val-b.val); for(ListNode node : lists) if(node!=null) pq.offer(node); while(!pq.isEmpty()) { ListNode curr = pq.poll(); tail.next = curr; if(curr.next!=null) pq.offer(curr.next); }</pre>	Time: O(N log K) Space: O(K)

1. FOUNDATIONS & TERMINOLOGY

- **Binary Tree:** Each node has at most 2 children (Left, Right).
- **Binary Search Tree (BST):** Left Subtree < Parent < Right Subtree.
- **Balanced Tree:** Height difference between left and right subtree ≤ 1 for all nodes.
- **Height:** Longest path from root to leaf.
- **Diameter:** Longest path between any two nodes.

2. TRAVERSALS & TREE RECURSION FLOW

Depth First Search (DFS):

- **Preorder (Root, Left, Right):** Used to clone, serialize, prefix paths.
- **Inorder (Left, Root, Right):** In BST, gives sorted order. Used to validate BST.
- **Postorder (Left, Right, Root):** Used when children must report answers first (Max Depth, Diameter, LCA).

Tree Recursion Template:

```
public int dfs(TreeNode root) {  
    if (root == null) return 0; // 1. Base case  
    int left = dfs(root.left);    // 2. Recurse left  
    int right = dfs(root.right); // 3. Recurse right  
    return Math.max(left, right) + 1; // 4. Combine  
}
```

3. BFS (LEVEL ORDER) PATTERNS

BFS Level Order Template:

```
Queue<TreeNode> q = new LinkedList<>();  
q.offer(root);  
while (!q.isEmpty()) {  
    int size = q.size();  
    for (int i = 0; i < size; i++) {  
        TreeNode curr = q.poll();  
        if (curr.left != null) q.offer(curr.left);  
        if (curr.right != null) q.offer(curr.right);  
    }  
}
```

Used for: *Right Side View, Zigzag Level Order.*

1. GRAPH REPRESENTATIONS

Type	Structure	Memory	Usage
Adjacency List	Map<Integer, List<Integer>>	O(V + E)	Most common for sparse graphs.
Adjacency Matrix	int[][] adj	O(V²)	Good for dense graphs, quick edge checks.
Grid (2D)	char[][] grid	O(M*N)	Cells are nodes, neighbors are 4 directions.

2. DFS VS BFS IN GRAPHS

- Breadth-First Search (BFS):**
- Uses Queue.
 - Guarantees **shortest path** in unweighted graphs.
- Depth-First Search (DFS):**
- Uses recursion / call stack.
 - Great for path finding, topological sort, cycle detection.
- Cycle Detection (Undirected):**
- Check if an adjacent node is visited and is NOT the parent of the current node.

3. TOPOLOGICAL SORT & DSU

- Topological Sort (Kahn's):**
- Calculate in-degree for all vertices.
 - Offer vertices with in-degree = 0 to a Queue.
 - Poll, add to topological order, and decrement neighbors' in-degree.
- Union-Find (DSU):**
- Maintains disjoint sets.
 - Operations: `find()` (with path compression) and `union()` (by rank/size).